My Habit Tracker
- Backend Project -

The My Habit Tracker app is a CLI-based habit tracker backend built in Python 3.7+.

Data will be stored in a local database file, which will then be used to analyze tracking data.

Design Approach

Modular architecture
- **Object oriented programming (OOP) paradigm: 3 classes**
- **Functional programming (FP) paradigm: 5 modules + 1 main file**

→ Each component has a clear and specific purpose
→ Advantages: code reusability, facilitated testing, better error handling
→ Differentiation between classes, helper functions, analytics and main modules

Database implementation:
- **Relational database**
- **Centralized database in one file**

→ File-based, fits well to modularity of project
→ Ensures consistency of data and error recovery
→ Supports SQL queries which facilitates data analysis

User interaction:
- **Inbuilt CLI**
- **User navigation by dialog/prompt structure**

→ Text-based menus with guided structure
→ Applicable for non-technical users
→ No need of complicated GUI

Tools

Error Handling

→ Use of Python's Try-except blocks and logging.basicConfig()
→ Helps in debugging and monitoring the program's execution
→ Used to track errors and program events

CLI: input()

→ Used for collecting user responses
→ Logical and well applicable navigation for the user
→ Easy to implement because of in-built function

Helper Functions

→ Shared logic: Functions managing the classes are implemented in extra modules to reduce redundancy and to keep the classes lightweight

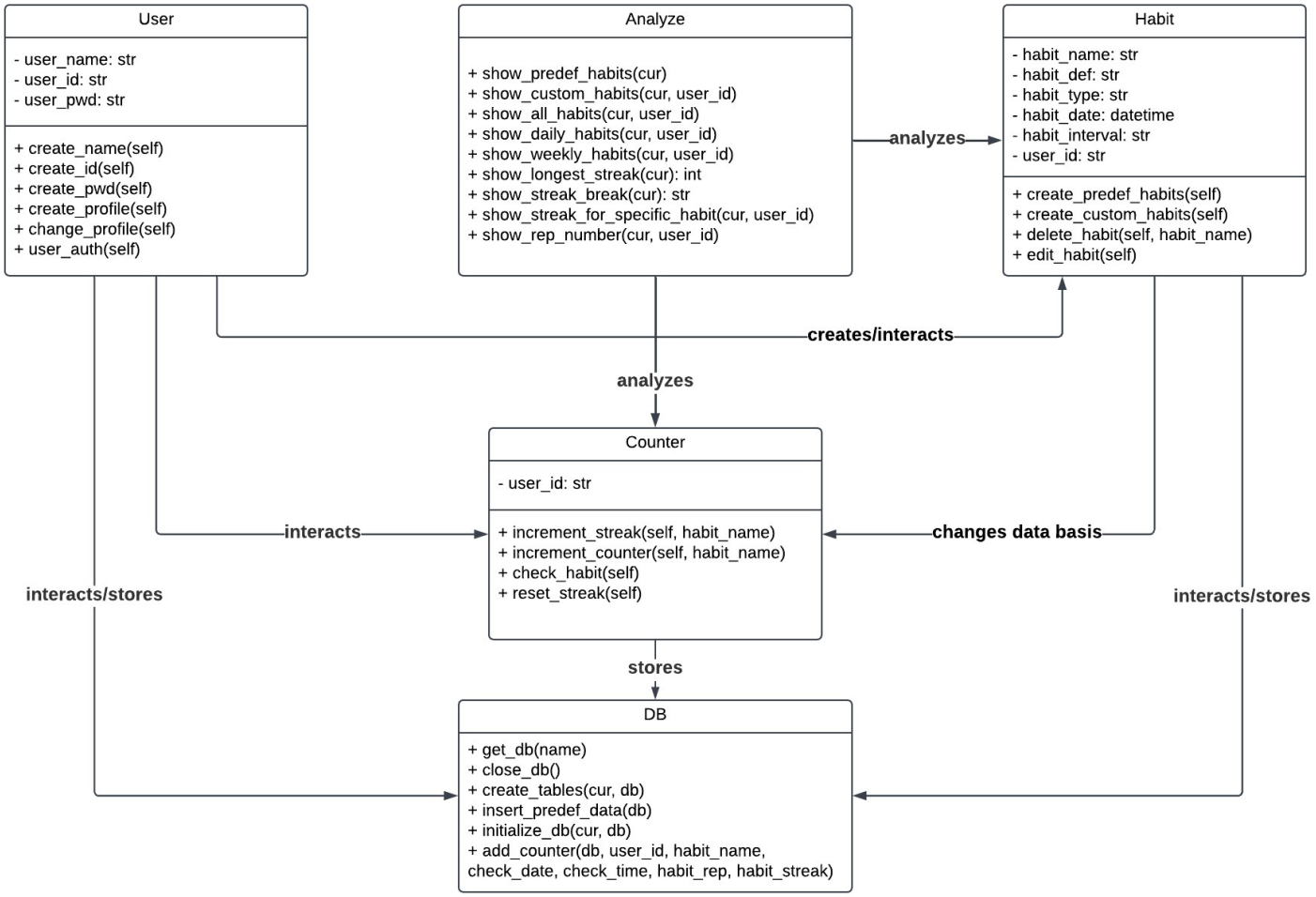
Password: getpass()

→ Securely promts a user password without displaying them on the screen for higher data security

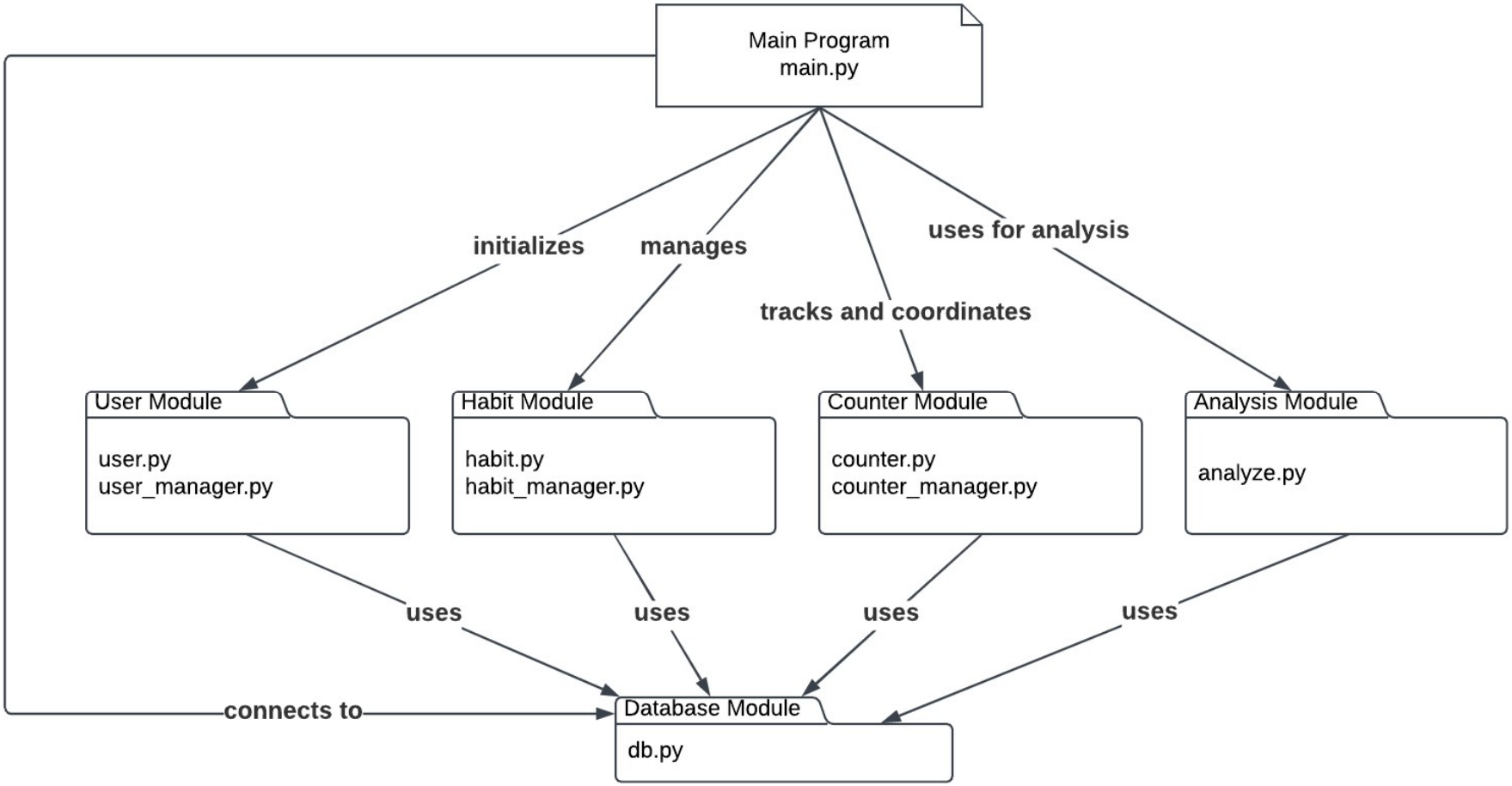
Libraries

Pandas	→ Organizing and structuring data into tables → Filtering and presenting data in a readable and user-friendly format	External (third-party)
SQLite	→ Used to structure relationships between modules using foreign keys → Lightweight, serverless database management tool that stores data in a local file → Easy integration in Python through built-in sqlite3 library	Internal (built-in)
Datetime	→ Recordings of creation date and tracking data → Easy calculations of streaks based on daily or weekly intervals → Ensures consistent date and time formatting	
Os	→ Used to check if the database file exists before attempting to connect or create it → Avoids errors during initialization and allows for conditional actions, like creating a new database if none exists	
Logging	→ Used for error handling: monitoring and warning (e.g., query failures) → Used to inform about program events (e.g., database initialization)	

UML Class Diagram



Package Diagram



User & Habit Modules

- User Class (OOP) and helper module User Manager (FP)
- Functionality:
  - User Class: User instances; methods call functions from user manager
  - User Manager: Functions to create, manage, and authenticate user profiles
  - Features:
    - ⇨ Password management
    - ⇨ Encouragement of unique user ID
    - ⇨ Guided user menu for editing
    - ⇨ Database interaction: storage and queries

User Module

- Habit Class (OOP) and helper module Habit Manager (FP)
- Functionality:
  - Class: Habit instances; methods call functions from habit manager
  - Habit Manager: Functions to create predefined and custom habits, delete and edit custom habits
  - Features:
    - ⇨ Customizability due to attributes as name, description, type, and interval
    - ⇨ Database interactions: create, read, update, delete operations performed in habit manager

Habit Module

Counter & Analytics Modules

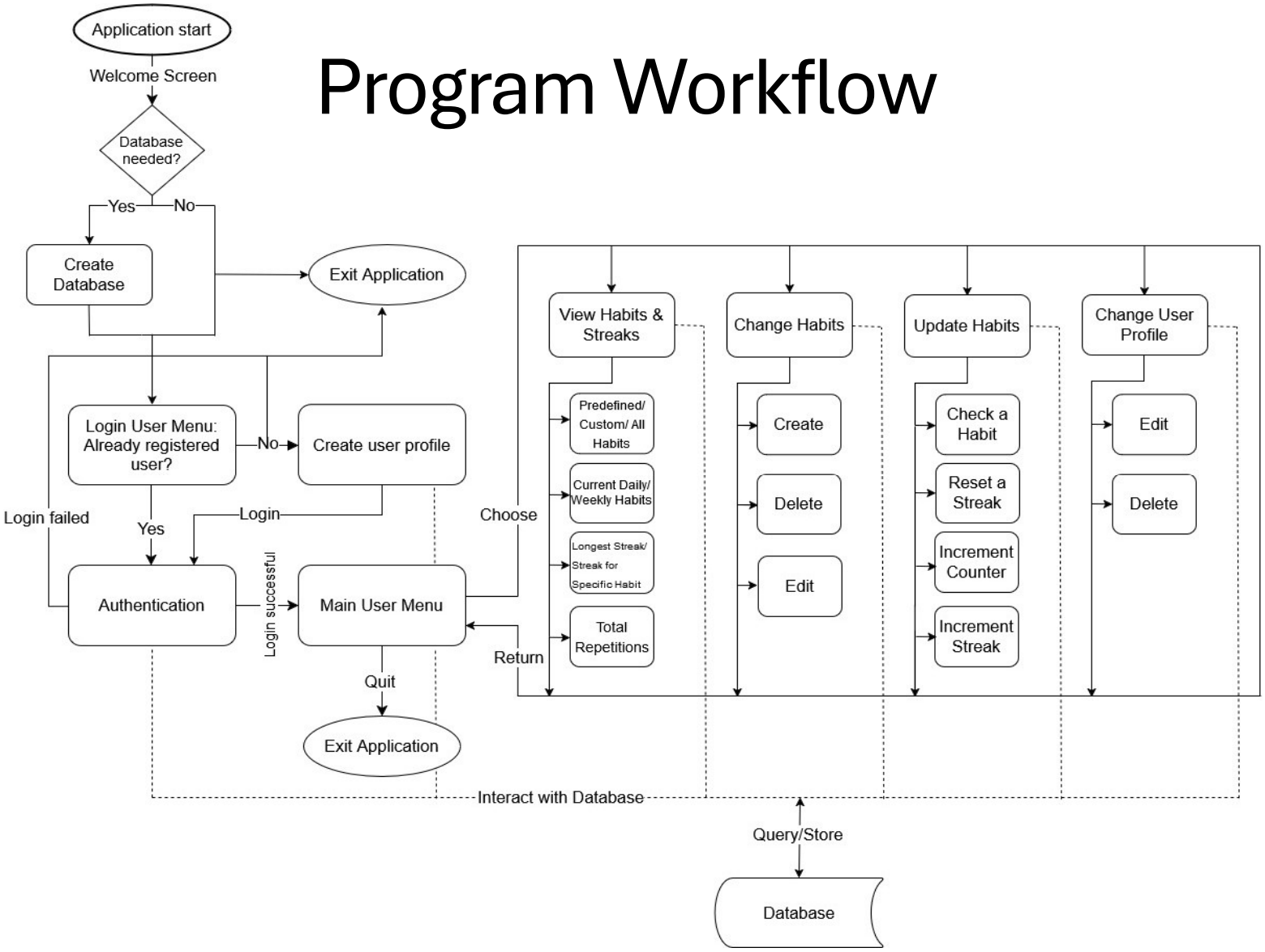
- Counter Class (OOP) and helper module Counter Manager (FP)
- Functionality:
  - Class: Counter instances; methods call functions from counter manager
  - Counter Manager: Functions to perform counter and streak incrementation, streak reset as well as the check-off of a specific habit
  - Features:
    - ⇨ Automatic vs. manual interaction: Check function increments counter and streak automatically, counter and streak can be incremented manually if user forgot to check the habit; streak can be reset manually
    - ⇨ Database interaction only possible if user enters habit name prior to data updates
    - ⇨ Database operations: Read, update
    - ⇨ Imports functions from analytics module to show all habits prior to data updates

Counter Module

- Analyze module (FP; no class): Provides insights into a user's progress and data
- Functionality: Allows users to ...
  - View and filter habits (predefined and custom)
  - Analyze habits based on intervals (daily or weekly)
  - Gain insights into streaks, number of repetitions, and broken streaks
  - Features:
    - ⇨ Uses pandas for organizing and manipulating fetched data into structured tables
    - ⇨ Database operations: Read, update
    - ⇨ Functions are called in main.py and in counter_manager.py

Analytics Module

Main Module

- Main module (FP; no class): Entry point and coordination of the Habit Tracker backend
- Procedure:
  - Welcome Menu
  - Database Initialization
  - User Authentication
  - Main Menu:
    - View habits and streaks
    - Change habits (create, edit, delete)
    - Update habits and streaks (check habits, increment counters, reset streaks)
    - Edit user profiles (update details or delete accounts)
- Features:
  - Interactive, menu-driven interface for users
  - Coordinates all modules and classes
  - Database operations: Read, update
  - Functions are called in main.py and in counter_manager.py

Program Workflow